PROJET 02

Mini EDR

Projet noyau Linux — Défense

AUTEUR
Lionel Auroux

ANNÉE
2026

GROUPE
4 étudiants

RENDU
Voir planning / Classroom

Sommaire

Sujet . . . 3
Mini EDR (Endpoint Detection and Response) . . . 3
1 - Rendu . . . 3
2 - Objectifs . . . 4
3 - Architecture imposée . . . 4
4 - Hooks BPF-LSM disponibles . . . 5
Exécution de processus . . . 5
Accès au système de fichiers . . . 5
Réseau . . . 5
Modules noyau . . . 6
Montage et stockage . . . 6
Mémoire et IPC . . . 6
5 - Fonctionnalités de votre projet . . . 7
Fonctionnalités obligatoires . . . 7
Fonctionnalités recommandées (non obligatoires) . . . 7
6 - Scénarios de démonstration . . . 7
Bonus : collaboration inter-groupes . . . 8
7 - Contraintes techniques . . . 8

Sujet

Mini EDR (Endpoint Detection and Response)

Vous devez réaliser un EDR minimal pour Linux par groupe de 4. La date de rendu est présente dans votre planning et sera programmée sur la classroom.

Le contexte d'exécution de l'EDR :

- La machine cible exécute un noyau Linux que vous aurez recompilé avec les options BPF requises (voir section Contraintes techniques).
- L'instrumentation du noyau repose **exclusivement** sur **BPF-LSM (KRSI)** : aucun module noyau (.ko) ne sera utilisé pour la partie détection ou blocage.

1 - Rendu

Vous aurez à scripter la fabrication d'un système Linux de test pour votre démonstration.

Vous rendrez une image QCOW2 de votre disque contenant votre LFS (Linux From Scratch) avec **votre propre kernel recompilé**. Utilisez `qemu-img convert -c ...` pour obtenir le fichier en QCOW2 compressé.

Vous serez pénalisé si le type d'image de votre rendu n'est pas le bon (QCOW2 compressé) ou si le kernel livré n'est pas compilé par vos soins avec les flags BPF-LSM requis.

Fichiers supplémentaires à rendre :

- Documentation utilisateur décrivant comment démarrer l'EDR, comment utiliser `edrctl` et comment interpréter les alertes.
- Rapport de conception de vos programmes. Votre rapport contiendra :
  - La répartition des tâches et le rôle de chacun des membres du groupe.
  - Un tableau de ratio (en pourcentage) du travail fourni sur l'ensemble du projet. Ce tableau sera utilisé pour la notation. Un travail normal d'un étudiant normal aura un ratio normal de 100%. La note du groupe multipliée par ce ratio donnera le total de points. Les fantômes à 0% auront 0 sur le projet. Vous êtes libre sur toutes les valeurs intermédiaires.
  - La justification de vos choix de hooks BPF-LSM et de mécanismes de réponse, en décrivant le scénario d'attaque que chaque fonctionnalité couvre.
  - Les scénarios de démonstration choisis, avec les étapes de test reproductibles.
- Sources codes :
  - Source du composant BPF (`edr.bpf.c`) et de son Makefile.
  - Source du démon userspace (`edrd`) et de son Makefile.
  - Source de l'outil de contrôle (`edrctl`) et de son Makefile.
  - Scripts de fabrication de la LFS et de recompilation du kernel.

  - Configuration du kernel (`.config` ou `defconfig` annoté).
- Votre LFS contiendra 2 comptes :
  - Un compte root permettant de démarrer le démon EDR et de démontrer les fonctionnalités de blocage.
  - Un compte user permettant de simuler un attaquant et de déclencher les alertes.
- Les login/mdp des différents comptes seront donnés dans la documentation.
- Le code devra respecter au maximum les consignes de l'ANSSI concernant le développement sécurisé (C Coding Standards) pour les parties en C. L'usage d'une langue différente de l'anglais pour les identifiants est proscrit. Pour la partie BPF, le style libbpf sera respecté (nommage en snake_case, `SEC()` corrects).

2 situations peuvent conduire à un 0 sans justification de ma part :

- Il est INTERDIT de remplacer BPF-LSM par ftrace, par modification de `sys_call_table` ou par tout autre mécanisme de hooking noyau. Soyez défensifs, pas créatifs sur ce point.
- Il est INTERDIT de rendre un EDR dont vous ne serez pas les auteurs. Tout code similaire entre différents groupes d'une même promo entraînera un 0 à tous les groupes.

2 - Objectifs

L'objectif de ce projet est de parfaire votre compréhension des domaines suivants :

- Système d'exploitation Linux et Kernel Linux,
- BPF / BPF-LSM (KRSI),
- Architecture d'un EDR (détection, réponse, reporting),
- Développement système en C avec libbpf et CO-RE,
- Recompilation du noyau Linux.

3 - Architecture imposée

Votre EDR sera composé de **trois éléments distincts** :

Composant	Binaire	Rôle
Agent BPF	`edr.bpf.c`	Programmes BPF-LSM hookant le noyau
Démon	`edrd`	Charge les BPF, lit le ring buffer, logue les événements
Outil de contrôle	`edrctl`	Interface CLI pour afficher les alertes et gérer les règles

La communication entre l'agent BPF et le démon se fait via un **BPF ring buffer**. La communication entre le démon et `edrctl` se fait via des **BPF maps pinnées** dans `/sys/fs/bpf/edr/` (listes noires, politiques de confinement, statistiques).

Le démon `edrd` doit démarrer automatiquement au boot de la LFS et écrire ses alertes dans `/var/log/edr.log`.

4 - Hooks BPF-LSM disponibles

Vous devez implémenter **au minimum 3 hooks** parmi la liste suivante. Chaque hook choisi doit être justifié dans votre rapport par le scénario d'attaque qu'il couvre.

Exécution de processus

Hook	Événement	Usage EDR
`bprm_check_security`	Exécution d'un binaire (execve)	Liste noire d'exécutables, détection de binaires suspects dans `/tmp`
`task_fix_setuid`	Appel à `setuid` / `seteuid`	Détection d'élévation de privilèges (sudo, exploitation)
`ptrace_access_check`	Tentative d'attache `ptrace`	Détection d'injection dans un processus, anti-debug
`task_kill`	Envoi d'un signal à un processus	Détection de tentatives de kill du démon EDR lui-même
`task_prctl`	Appel à `prctl()`	Détection de modification de capabilities ou de dumpable

Accès au système de fichiers

Hook	Événement	Usage EDR
`file_open`	Ouverture d'un fichier	Surveillance d'accès à des fichiers sensibles (`/etc/shadow`, clés SSH)
`inode_create`	Création d'un fichier	Détection de dépôt de fichiers suspects dans des répertoires système
`inode_unlink`	Suppression d'un fichier	Détection d'effacement de logs ou de traces d'intrusion
`inode_rename`	Renommage d'un fichier	Détection de remplacement de binaires système
`inode_setattr`	Modification d'attributs (chmod, chown)	Détection d'un `setuid` posé sur un binaire

Réseau

Hook	Événement	Usage EDR

`socket_connect`	Connexion réseau sortante	Détection de C2 (Command & Control), reverse shell
`socket_bind`	Écoute sur un port	Détection d'ouverture de backdoor ou de bind shell
`socket_listen`	Mise en écoute active	Détection de serveurs non autorisés
`socket_sendmsg`	Envoi de données	Détection d'exfiltration de données

Modules noyau

Hook	Événement	Usage EDR
`kernel_module_request`	Demande de chargement d'un module	Blocage de LKM rootkits (comme ceux du projet 01)
`kernel_load_data`	Chargement de données noyau	Détection de firmware non signé ou de BPF malveillant

Montage et stockage

Hook	Événement	Usage EDR
`sb_mount`	Montage d'un système de fichiers	Détection de montage de périphériques amovibles en cours d'examen
`sb_umount`	Démontage	Détection d'effacement de preuves via démontage

Mémoire et IPC

Hook	Événement	Usage EDR
`mmap_file`	Mapping d'un fichier en mémoire exécutable	Détection de fileless malware (`memfd_create` + exec)
`shm_shmat`	Attachement de mémoire partagée	Détection de communication inter-processus suspecte

5 - Fonctionnalités de votre projet

Fonctionnalités obligatoires

- **Au moins 3 hooks BPF-LSM** actifs parmi la liste ci-dessus.
- **Au moins 1 mécanisme de réponse** : blocage actif (retour `-EPERM`) ou confinement de processus via BPF map.
- **Ring buffer** pour remonter les événements au démon `edrd`.
- **Logging** : chaque événement loggé contient horodatage, PID, UID, `comm` du processus et le détail de l'événement (chemin, adresse réseau, etc. selon le hook).
- `edrctl` avec au minimum les sous-commandes :
  - `edrctl status` : état du démon et liste des hooks actifs.
  - `edrctl watch` : affichage en temps réel des alertes.
  - `edrctl block add <chemin|ip>` : ajout d'une règle de blocage.
  - `edrctl block list` : liste des règles actives.

Fonctionnalités recommandées (non obligatoires)

- Confinement de processus (jail) : restreindre les droits d'un PID via une BPF map de politique, sans le tuer.
- Statistiques par processus (compteur d'événements suspects par PID).
- Rotation des logs et limite de taille de `/var/log/edr.log`.

6 - Scénarios de démonstration

La LFS doit permettre de démontrer **au moins 2 scénarios reproductibles** décrits dans votre documentation. Chaque scénario précise :

- L'action simulée par le compte `user` (l'attaquant).
- L'alerte ou le blocage produit par l'EDR (visible dans `edrctl watch` ou le log).
- La commande de vérification permettant de valider le comportement.

Exemples de scénarios possibles (non exhaustifs) :

- Tentative d'exécution d'un binaire présent dans la liste noire → blocage + alerte.
- Lecture de `/etc/shadow` par un processus non privilégié → alerte.
- Connexion vers une IP suspecte → alerte ou blocage.
- Tentative de chargement d'un module `.ko` → blocage + alerte.
- Tentative de tuer le démon `edrd` via `kill` → alerte ou blocage.

Bonus : collaboration inter-groupes

Un groupe EDR peut se rapprocher d'un groupe ayant réalisé le **projet Rootkit** pour démontrer que leur EDR détecte les techniques offensives du rootkit (masquage de fichiers, persistance, communication avec le programme compagnon). Cette démonstration croisée est valorisée comme bonus, à condition que les deux groupes soient clairement identifiés dans leurs rapports respectifs.

7 - Contraintes techniques

- **BPF-LSM obligatoire** : aucun usage de `sys_call_table`, de ftrace ou de tout autre mécanisme de hooking noyau pour la partie détection/blocage.
- **Recompilation du kernel** : votre LFS doit embarquer un kernel compilé par vos soins avec au minimum :
  - `CONFIG_BPF=y`
  - `CONFIG_BPF_SYSCALL=y`
  - `CONFIG_BPF_LSM=y`
  - `CONFIG_LSM="...,bpf"` (bpf ajouté à la liste des LSM actifs)
  - `CONFIG_DEBUG_INFO_BTF=y` (nécessaire pour CO-RE)
- **Outillage** : libbpf + CO-RE. L'utilisation de BCC ou d'un autre framework BPF doit être justifiée.
- **BPF maps pinnées** : les maps de politique doivent être pinnées dans `/sys/fs/bpf/edr/` pour permettre à `edrctl` d'y accéder sans passer par le démon.
- **Pas de désactivation de SELinux/AppArmor** dans les scripts de la LFS pour simplifier les tests : si votre LFS en embarque un, configurez-le correctement ou justifiez son absence.